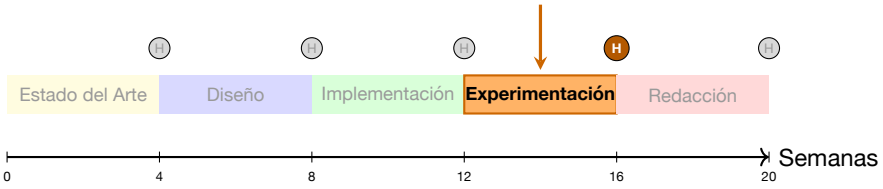
Documento de mes 4 — Experimentación



Experimentación: Escalado, Diversidad, Estabilidad y Validación

TFM: Selección Automática de Características en Algoritmos de Clasificación Mediante Algoritmos Genéticos y Métodos de Envoltorio con Apache Spark RDD

Este documento recoge los hitos 7 a 12 del plan de trabajo: desde la incorporación de los datasets Arrhythmia y Gisette (Hito 7) hasta el análisis de escalabilidad paralela (Hito 12), pasando por la refactorización del núcleo del GA, el estudio de diversidad genética, el estudio de estabilidad y la validación del subconjunto final con LinearSVC.

Documentos internos referenciados

Abrev.	Fichero	Descripción
[D1]	`diseño_GA.md`	Diseño del GA: fitness §2, operadores §3, clasificadores §5, parámetros §6
[D2]	`arquitectura_spark_rdd_ga.md`	Arquitectura Spark RDD: Driver/Workers §2.1, broadcast §2.5
[D4]	`plan_diversidad_escalabilidad.md`	Marco conceptual y plan experimental: métricas de diversidad §1.2, estabilidad §3.2, sesgo de selección y holdout §4

Resumen de hitos cubiertos

Hito	Título	Notebook / Módulo
7	Incorporación de Arrhythmia y Gisette	`05_ga_full_arrhythmia.ipynb`, `06_ga_full_gisette.ipynb`
8	Refactorización: GA-wrapper reutilizable	`src/ga_core.py`, `src/ga_metrics.py`
9	Análisis de diversidad genética	`07_diversidad_escalabilidad.ipynb` §1
10	Estudio de estabilidad ($R = 10$ semillas)	`07_diversidad_escalabilidad.ipynb` §2
11	Validación con LinearSVC	`07_diversidad_escalabilidad.ipynb` §3
12	Análisis de escalabilidad paralela	`07_diversidad_escalabilidad.ipynb` §4

---

1. Hito 7: Incorporación de Arrhythmia y Gisette

1.1 Objetivo

Extender el GA completo —validado sobre WDBC en el mes 3— a dos datasets de mayor dimensionalidad: **Arrhythmia** ($D = 279$, multiclase) y **Gisette** ($D = 5000$, binario), cubriendo así el espectro de complejidad previsto en el diseño del TFM (**[D1]** §6).

1.2 Descripción de los datasets

Dataset	N	D	Clases	Espacio	Fuente / Preprocesado
WDBC	569	30	2	2^{30}	`sklearn.datasets` / ninguno
Arrhythmia	452	279	13	2^{279}	OpenML `arrhythmia` v1 (Guvenir et al., 1997) / imputación por media (408 valores)
Gisette	7000	5000	2	2^{5000}	OpenML `gisette` (Guyon et al., 2004) / ninguno (ya escalado)

Arrhythmia presenta una distribución de clases altamente desequilibrada: la clase mayoritaria (*normal*) agrupa 245 de las 452 muestras, mientras que varias clases minoritarias cuentan con 2–5 muestras (clases 7, 8, 14, 15). Esta propiedad obligó a adaptar el número de pliegues de validación cruzada de 5 a 3, ya que con 5 pliegues algunos pliegues quedan sin representación de las clases más raras (`StratifiedKFold` emite aviso de clase con menos muestras que el número de pliegues). Esta decisión se documenta explícitamente en el notebook (`05_ga_full_arrhythmia.ipynb`, §2.1) y no afecta la comparabilidad con WDBC, pues el número de pliegues forma parte de los parámetros adaptados por dataset en **[D1]** §6.

Gisette, por su parte, tiene $D = 5000$ características con la mayoría irrelevantes (variables aleatorias mezcladas con 5 dígitos manuscritos relevantes), lo que justifica una probabilidad de inicialización $p_{\text{init}} = 0.1$ frente al 0.5 del resto de datasets, generando cromosomas iniciales con ~500 genes activos en lugar de ~2500.

1.3 Adaptación de parámetros por dataset

Parámetro	WDBC	Arrhythmia	Gisette
Tamaño población (P)	50	100	100
Máx. generaciones (G_{max})	50	100	100
Pliegues CV (k)	5	3	5
Prob. inicialización (p_{init})	0.5	0.5	0.1
Elitismo (E)	2	5	5
Ventana parada (N_{stop})	10	15	15
$p_{\text{mut_bit}}$	$1/30$	$1/279$	$1/5000$

Los parámetros comunes ($\alpha = 0.9$, $\beta = 0.1$, $p_{\text{cross}} = 0.8$, $p_{\text{mut}} = 0.2$, $k_{\text{tourn}} = 5$, $\varepsilon_{\text{stop}} = 0.001$) se mantienen iguales en los tres datasets para garantizar comparabilidad entre experimentos (**[D1]** §6). La elección de GNB como clasificador base durante la búsqueda sigue la práctica establecida en la literatura de GA-wrapper (Aalaei et al., 2016; Huang et al., 2020; Rashid & Abdullah, 2014), mientras que la selección de GAs para explorar el espacio 2^D de subconjuntos —NP-completo (Guyon & Elisseeff, 2003; Siedlecki & Sklansky, 1989)— está justificada por su capacidad de exploración eficiente.

1.4 Resultados del GA (run único, semilla 42, dataset completo)

Los notebooks 05 y 06 ejecutan el GA sobre el **dataset completo** (sin holdout), con semilla fija para reproducibilidad. Los resultados se exportan en formato JSON (`ga_full_arrhythmia_experimento.json`, `ga_full_gisette_experimento.json`).

Dataset	Gens.	Motivo parada	$\lvert S^* \rvert$	Reducción	Fitness
WDBC	35	Convergencia	6/30	80%	0.861943
Arrhythmia	100	G_{max} alcanzado	93/279	66.7%	0.189779
Gisette	44	Convergencia ($\Delta = 0.0009$)	486/5000	90.3%	0.823809

Arrhythmia no converge en $G_{\text{max}} = 100$ generaciones (Figura~1). La función de fitness muestra mejoras marginales y sostenidas hasta la última generación (gen 98: 0.189421 $\rightarrow$ gen 100: 0.189779), sin alcanzar el criterio de estancamiento ($\Delta < 0.001$ en 15 generaciones). Esto es consistente con la dificultad del problema: 13 clases, alto desequilibrio y 279 variables candidatas generan un espacio de búsqueda donde el GA no logra estabilizar un óptimo en 100 generaciones.

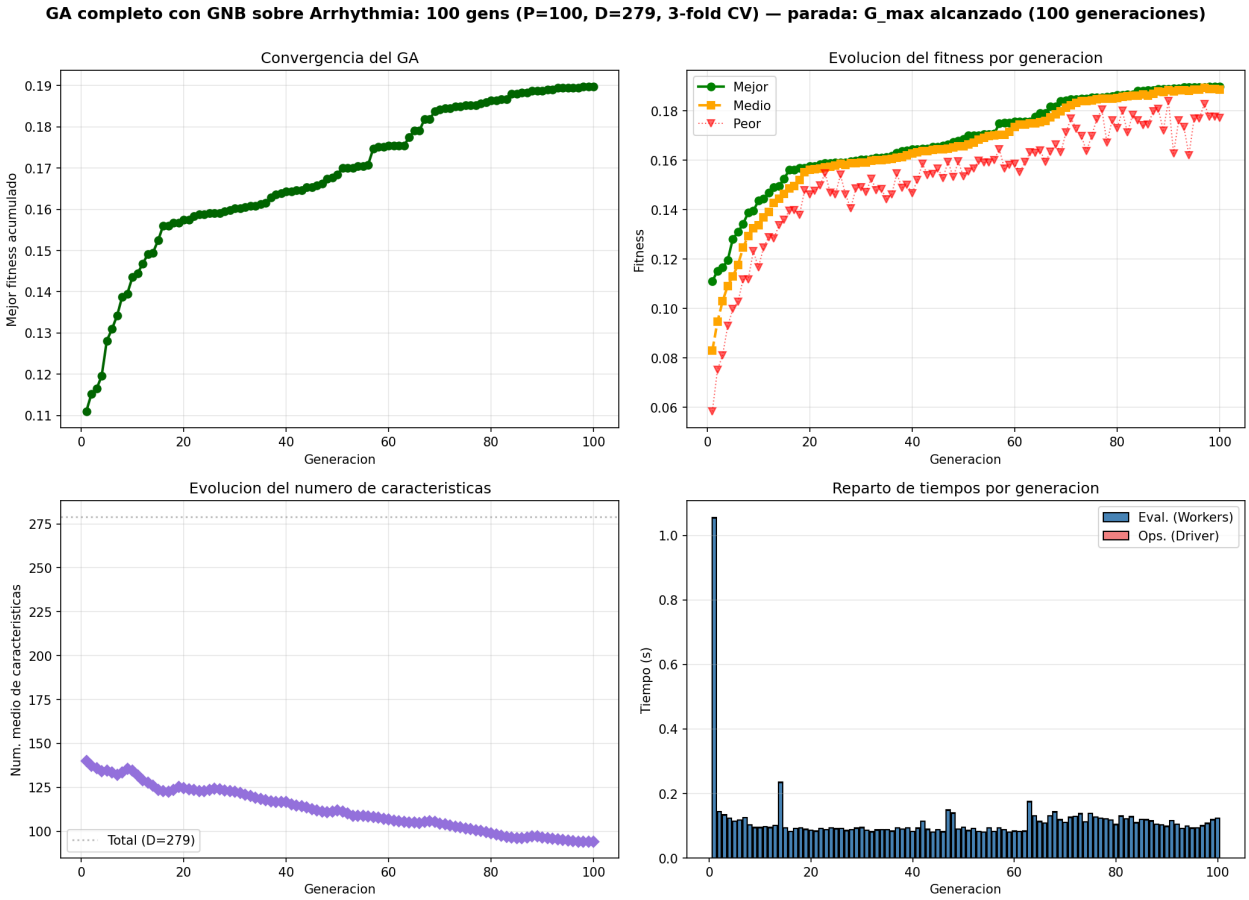


Figure 1: GA completo sobre Arrhythmia ($P = 100$, $D = 279$, CV-3, semilla 42, dataset completo). De izquierda a derecha y de arriba a abajo: (1) mejor fitness acumulado por generación (convergencia monótona); (2) evolución del mejor, medio y peor fitness por generación; (3) número medio de características activas por generación, decreciente de $\sim$140 a $\sim$94; (4) reparto de tiempos por generación entre evaluación Spark (Workers, azul, $\sim$0.10 s) y operadores genéticos (Driver, rojo, < 0.001 s). El GA alcanza $G_{\text{max}} = 100$ generaciones sin converger, con fitness final 0.1898 y $|S^*| = 93/279$.

Nota metodológica sobre la interpretación del fitness en Arrhythmia. El valor 0.189779 corresponde al fitness calculado sobre el dataset completo con CV-3:

$$F = 0.9 \times \text{acc}_{CV3} - 0.1 \times \frac{|S|}{D} = 0.9 \times 0.2479 - 0.1 \times \frac{93}{279} \approx 0.190$$

La accuracy de ~24.8% sobre el dataset completo refleja la dificultad intrínseca de GNB ante 13 clases desequilibradas, no un mal funcionamiento del GA. En los experimentos posteriores con holdout (§3), el particionado estratificado fallido (clases de 2 muestras no estratificables) produce una escisión no estratificada que excluye las clases más raras del entrenamiento, resultando en un fitness de entrenamiento aparentemente más alto (0.567) que no debe interpretarse como mejora real del algoritmo, sino como un artefacto del split.

Gisette converge en solo 44 generaciones (Figura~2), lo que resulta llamativo dado su enorme espacio de búsqueda (2^{5000}). La explicación reside en la estructura del problema: la mayoría de las 5000 variables son ruido, y GNB detecta rápidamente un subconjunto relevante de ~486 características que captura casi

toda la información discriminativa. La presión selectiva con $k_{\text{tourn}} = 5$ y elitismo $E = 5$ concentra la población en torno a ese subconjunto en pocas generaciones, alcanzando el criterio de convergencia.

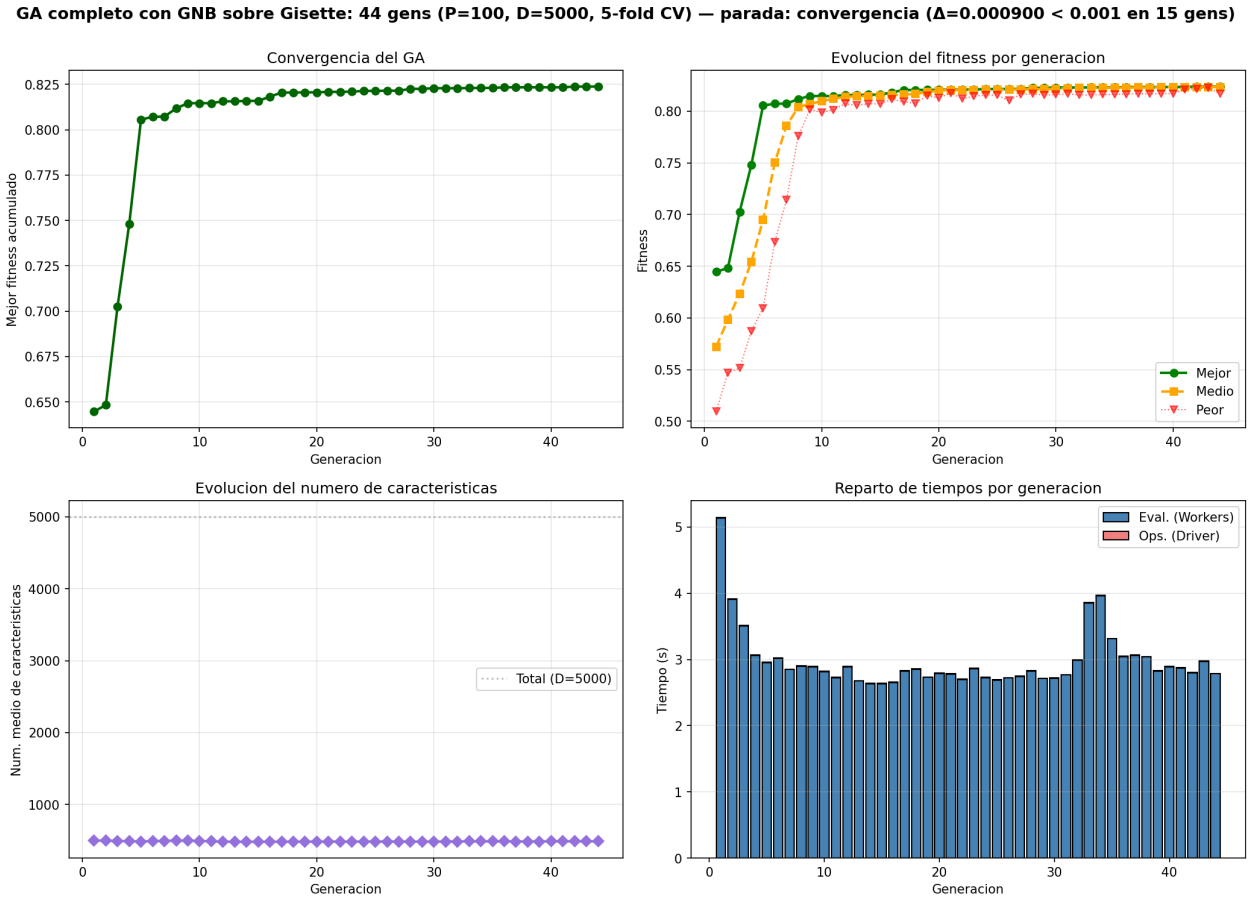


Figure 2: GA completo sobre Gisette ($P = 100$, $D = 5000$, CV-5, semilla 42, dataset completo). Los cuatro paneles equivalen a los de la Figura 1. Aspectos destacados: (1) convergencia rápida en 44 generaciones ($\Delta = 0.0009 < 0.001$ en 15 generaciones); (2) fitness medio muy próximo al mejor desde la generación 10, indicando homogeneización rápida de la población; (3) número medio de características estabilizado en ∼487/5000 ($p_{\text{init}} = 0.1$, cromosomas dispersos); (4) tiempo de evaluación Spark dominante (∼2.9 s/gen) frente a operadores (< 0.01 s/gen), relación 290:1 que justifica plenamente la paralelización.

---

2. Hito 8: Refactorización — GA-Wrapper Reutilizable

2.1 Motivación

Los notebooks 04, 05 y 06 contenían bucles evolutivos casi idénticos, con pequeñas variaciones por dataset. Esta replicación introducía riesgo de inconsistencias y dificultaba el estudio comparativo multi-dataset del mes 4. Se refactorizó el núcleo en dos módulos de `src/`:

- `ga_core.py`: función `run_ga()` que encapsula el ciclo evolutivo completo.
- `ga_metrics.py`: funciones de diversidad y estabilidad de la población.

2.2 Interfaz de `run_ga()`

```
def run_ga(X, y, params, seed=42, evaluador_poblacion=None,
           collect_diversity=True, verbose=False):
    """Ejecuta el GA-wrapper completo.

    params keys: P, G_max, p_cross, p_mut, p_mut_bit, k_tourn, elite_size,
                 p_init, eps_stop, N_stop, alpha, beta, cv_folds.
    evaluador_poblacion : callable(pop) -> list[float].
        Si None, evalúa en serie con GNB (mismo resultado que el evaluador
        Spark, que es determinista).

    Devuelve dict con mejor_cromosoma, mejor_fitness, mejor_n_caracteristicas,
    generaciones_ejecutadas, motivo_parada, series y tiempos.
    """
```

El parámetro `evaluador_poblacion` permite **inyectar el evaluador Spark RDD desde el notebook** sin que `run_ga` dependa de Spark. Esto facilita pruebas unitarias en serie y la comparación de tiempos entre evaluadores. La semilla se fija de forma reproducible: primero `np.random.seed(seed)` para la población inicial, luego `random.seed(seed)` para los operadores genéticos, en el mismo orden que los notebooks originales.

2.3 Módulo `ga_metrics.py`

Función	Descripción
`diversity_metrics(pop)`	Hamming media normalizada, entropía media por locus, fracción de individuos únicos
`jaccard_mean(subsets)`	Jaccard medio sobre todos los pares de subconjuntos ($R \times R$ matriz)
`kuncheva_index(subsets, D)`	Índice de Kuncheva: corrección por azar del solapamiento de subconjuntos
`selection_frequency(subsets)`	Fracción de ejecuciones en que cada característica aparece en S^*

La separación en módulos permite que `07_diversidad_escalabilidad.ipynb` use directamente estas funciones sin duplicar código, y que los workers de Spark las serialicen con `cloudpickle` (registradas con `register_pickle_by_value`) para garantizar que la función de fitness viaja embebida en cada tarea, sin dependencia de importaciones en los workers.

---

3. Hito 9: Análisis de Diversidad Genética

3.1 Métricas implementadas

La diversidad se evalúa con tres métricas complementarias calculadas en cada generación sobre la población completa (**[D4]** §1.2):

Métrica	Fórmula	Interpretación
Hamming media norm.	$\bar{H} = \frac{1}{D\binom{P}{2}} \sum_{i<j} d_H(\mathbf{c}_i, \mathbf{c}_j)$	Diversidad genotípica media; 0 = convergencia total, 1 = máxima dispersión
Entropía media por locus	$\bar{S} = \frac{1}{D} \sum_{l=1}^{D} H_b(p_l)$	Shannon por bit; captura heterogeneidad locus a locus
Fracción de únicos	$u = \lvert\{c : c \in \text{pop}\}\rvert / P$	Proporción de cromosomas distintos; 1 = sin duplicados

Las tres métricas se incluyen en el campo `series` que devuelve `run_ga()` y se exportan a los CSV de series por generación (`*_holdout_series.csv`).

3.2 Protocolo experimental

El estudio se realiza sobre el **split holdout** (70/30, `random_state=42`) con la semilla principal ($R_0 = 42$), con el parámetro `collect_diversity=True`. El holdout es necesario para que la validación del subconjunto S^* en test sea independiente del entrenamiento del GA (**[D4]** §4.3).

3.3 Resultados

Dataset	Gens.	$\bar{H}$ gen.1	$\bar{H}$ última	u gen.1	u última	Gen. $\bar{H} < 0.1$
WDBC	24	0.493	0.009	1.00	0.12	6
Arrhythmia	100	0.495	0.005	1.00	0.23	9
Gisette	58	0.178	0.0003	1.00	0.28	8

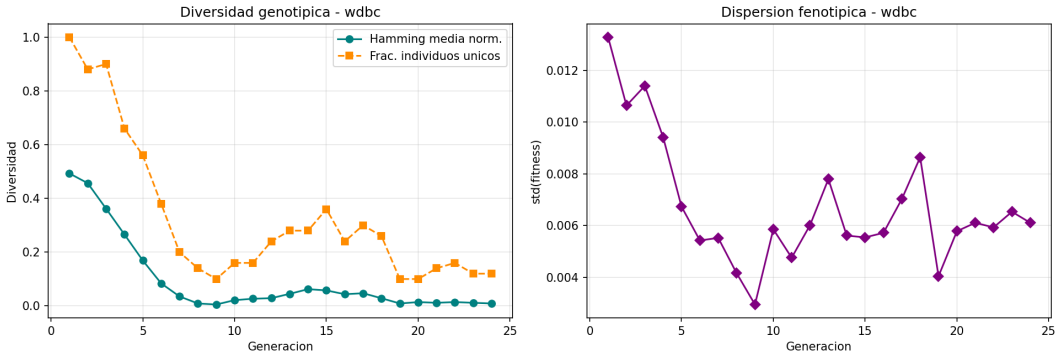


Figure 3: Diversidad genética y dispersión fenotípica — WDBC ($D = 30$, 24 generaciones). *Izquierda:* Hamming media normalizada (teal) y fracción de individuos únicos (naranja). La Hamming colapsa por debajo de 0.1 en la generación 6; la fracción de únicos exhibe oscilaciones residuales hasta el final. *Derecha:* desviación típica del fitness (dispersión fenotípica), que cae de $\sim$0.013 en la generación 1 y se estabiliza en torno a 0.005–0.007, con algún repunte por eventos de cruce favorables.

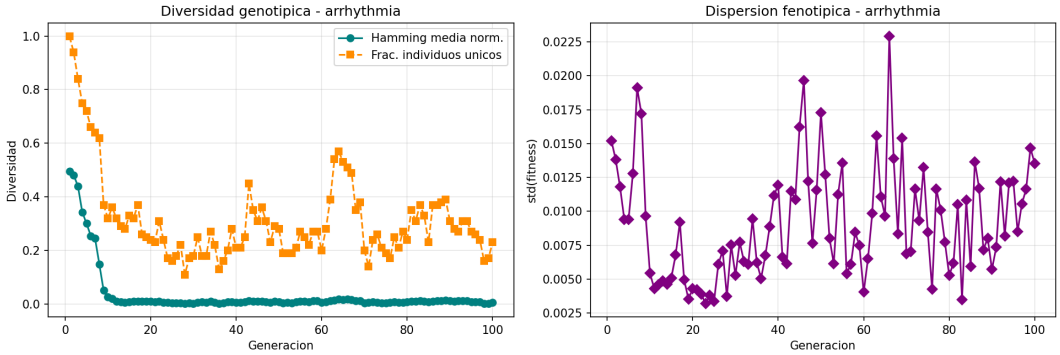


Figure 4: Diversidad genética y dispersión fenotípica — Arrhythmia ($D = 279$, 100 generaciones). *Izquierda:* la Hamming cae bruscamente en las primeras 9 generaciones y se mantiene cerca de 0 el resto de la evolución; la fracción de únicos oscila entre 0.1 y 0.5 durante las 100 generaciones, señalando que persiste cierta renovación genotípica por mutación aunque sin diversidad real. *Derecha:* la dispersión fenotípica (std fitness) muestra picos irregulares a lo largo de toda la ejecución —incluyendo un máximo pronunciado en torno a la generación 65— coherentes con una búsqueda que no converge y en la que eventos de mutación generan individuos de fitness muy variado.

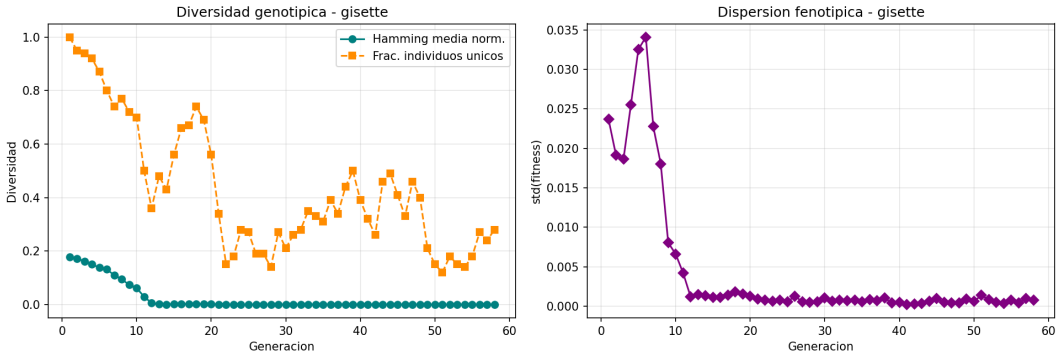


Figure 5: Diversidad genética y dispersión fenotípica — Gisette ($D = 5000$, $p_{\text{init}} = 0.1$, 58 generaciones). *Izquierda:* la Hamming parte de 0.178 (menor que en WDBC y Arrhythmia por la inicialización dispersa) y cae a cero en torno a la generación 8; la fracción de únicos oscila pero sin mostrar tendencia tras la convergencia genómica. *Derecha:* la dispersión fenotípica presenta un pico agudo entre las generaciones 5–7 —justo cuando el GA transita de exploración a explotación— y luego colapsa a valores próximos a cero, reflejando la alta homogeneidad fenotípica del estado convergido.

3.4 Interpretación estadística

Los tres datasets siguen un patrón de decaimiento monotónico, visible en las Figuras~3–5: la diversidad máxima en generación 1 (fracción de únicos = 1.0 en los tres casos, toda la población es distinta) colapsa rápidamente hacia la convergencia. En WDBC y Arrhythmia, la Hamming inicial de ~0.49–0.50 refleja la inicialización equiprobable ($p_{\text{init}} = 0.5$): con bits independientes de Bernoulli(0.5), la distancia de Hamming esperada entre dos cromosomas es exactamente $0.5D$, normalizada a 0.5.

Gisette comienza con $\bar{H} = 0.178$, significativamente menor, como consecuencia directa de $p_{\text{init}} = 0.1$: con cromosomas que activan solo el 10% de bits, la distancia de Hamming máxima posible entre dos cromosomas es proporcional a la densidad de bits activos, y la superposición de unos en regiones distintas produce distancias más pequeñas que con $p_{\text{init}} = 0.5$.

La velocidad de convergencia es llamativamente similar en los tres datasets (Figura~6): la Hamming cae por debajo de 0.1 entre las generaciones 6 y 9, con independencia de D (30, 279, 5000). Esto sugiere que la presión selectiva del torneo ($k = 5$) domina sobre la diversificación de la mutación ($p_{\text{mut_bit}} = 1/D$), y que el GA "decide" rápidamente en qué región del espacio explorar, independientemente de la dimensionalidad.

La fracción de únicos al final del GA es mayor en Gisette (0.28) que en WDBC (0.12), a pesar de que Gisette tiene menor Hamming final. Esto indica que en Gisette coexisten múltiples cromosomas casi idénticos en las características relevantes (bits 1 coincidentes) pero con pequeñas variaciones en las irrelevantes (bits 0 con algún ruido de mutación). WDBC, con solo 30 bits, colapsa hacia clones casi exactos.

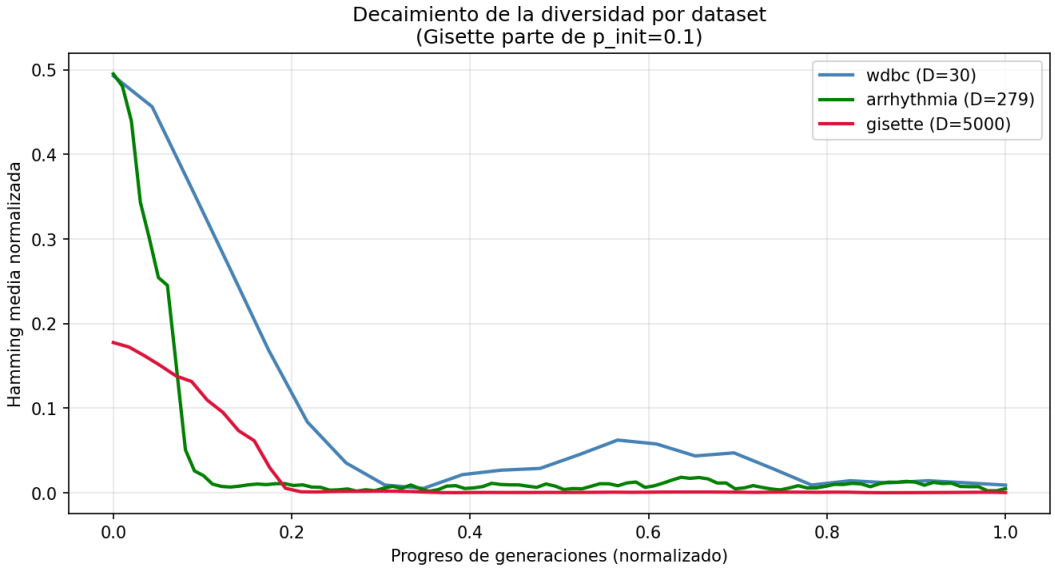


Figure 6: Decaimiento comparativo de la Hamming media normalizada en los tres datasets, con el progreso de generaciones normalizado al intervalo $[0, 1]$ para permitir la comparación directa. Gisette (rojo) parte de 0.178 por su inicialización dispersa ($p_{\text{init}} = 0.1$) y colapsa antes del 20% del ciclo; Arrhythmia (verde) y WDBC (azul), con $p_{\text{init}} = 0.5$, parten de $\sim$0.49 y siguen decaimientos similares, aunque WDBC muestra una convergencia algo más lenta en términos relativos (mayor persistencia de diversidad residual entre el 30% y el 60% del ciclo, posiblemente por el menor tamaño de población $P = 50$). Los tres datasets alcanzan Hamming ≈ 0 antes del 20% del ciclo normalizado.

---

4. Hito 10: Estudio de Estabilidad ($R = 10$ Semillas)

4.1 Protocolo

Se repite la ejecución del GA $R = 10$ veces con semillas consecutivas (42, 43, …, 51), manteniendo fijos el dataset (split holdout 70/30, `random_state`=42) y todos los hiperparámetros. Para cada ejecución se registran: subconjunto óptimo S^*, fitness, $|S^*|$ y accuracy del clasificador GNB sobre el test held-out. Tras las R ejecuciones se calculan:

- **Media y desviación típica** de accuracy, fitness y $|S^*|$ entre semillas.
- **Jaccard medio** entre todos los pares de subconjuntos:

$$J_{\text{medio}} = \frac{2}{R(R-1)} \sum_{i<j} \frac{|S_i^* \cap S_j^*|}{|S_i^* \cup S_j^*|}$$

- **Índice de Kuncheva** (Kuncheva, 2007; **[D4]** §3.2):

$$KI = \frac{1}{\binom{R}{2}} \sum_{i<j} \frac{|S_i^* \cap S_j^*| - \frac{|S_i^*| \cdot |S_j^*|}{D}}{\min(|S_i^*|, |S_j^*|) - \frac{|S_i^*| \cdot |S_j^*|}{D}}$$

- **Núcleo estable**: número de características con frecuencia de selección ≥ 0.8 (aparecen en ≥ 8 de 10 ejecuciones).

4.2 Resultados por dataset

Dataset	Acc. media	Acc. std	Fitness medio	Fitness std	$\lvert S^* \rvert$ medio	$\lvert S^* \rvert$ std
WDBC	0.9480	0.0108	0.8598	0.0065	5.7	1.77
Arrhythmia	0.3294	0.1892	0.3688	0.1272	105.4	10.19
Gisette	0.9199	0.0067	0.8187	0.0047	528.5	27.55

Dataset	Jaccard	Kuncheva	Núcleo ($\geq$0.8)	CV(acc)	CV(fitness)
WDBC	0.4265	0.6524	3	1.1%	0.8%
Arrhythmia	0.2975	0.1434	15	57.4%	34.5%
Gisette	0.0610	0.0108	0	0.7%	0.6%

Los datos por semilla permiten observar la dispersión individual (fuente: `estabilidad_por_semilla.csv`):

Semilla	WDBC acc	Arrhythmia acc	Gisette acc	Arrhythmia fitness
42	0.9474	0.5662	0.9210	0.5673
43	0.9298	0.5441	0.9205	0.5407
44	0.9532	0.1471	0.9148	0.2458
45	0.9649	0.4853	0.9338	0.4195
46	0.9532	0.1471	0.9233	0.2335
47	0.9357	0.4338	0.9129	0.4203
48	0.9532	0.1618	0.9210	0.2808
49	0.9415	0.5000	0.9252	0.4549
50	0.9415	0.1544	0.9119	0.2754
51	0.9591	0.1544	0.9143	0.2500

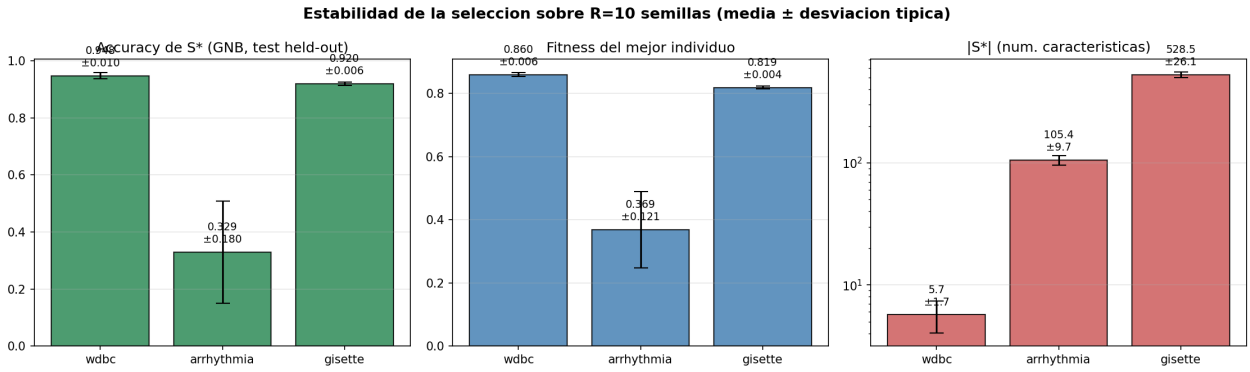


Figure 7: Estabilidad de la selección sobre $R = 10$ semillas (42–51): media $\pm$ desviación típica de accuracy GNB en test held-out (izquierda), fitness del mejor individuo (centro) y $\lvert S^* \rvert$ en escala logarítmica (derecha). WDBC y Gisette muestran barras de error muy estrechas (alta estabilidad fenotípica); Arrhythmia presenta una barra de error que abarca casi todo el rango posible de accuracy, evidenciando la bimodalidad descrita en el texto. El eje logarítmico del tercer panel permite comparar $\lvert S^* \rvert$ entre datasets con órdenes de magnitud distintos (3–9 en WDBC, 82–121 en Arrhythmia, 483–571 en Gisette).

4.3 Análisis estadístico

La Figura~7 resume visualmente los tres indicadores. **WDBC: estabilidad alta en accuracy, estabilidad moderada en identidad de subconjunto.**
El coeficiente de variación (CV) de la accuracy es del 1.1% y del fitness del 0.8%, indicando que el GA encuentra sistemáticamente soluciones de calidad equivalente. El rango del $\lvert S^* \rvert$ oscila entre 3 y 9 características, con Jaccard=0.427 y Kuncheva=0.652. Este último valor, próximo a 1, indica un solapamiento significativamente superior al azar. El núcleo estable de 3 características (frecuencia ≥ 0.8) identifica las variables más discriminativas del dataset: en WDBC con $D = 30$, el GA converge hacia distintos subconjuntos de tamaño pequeño que comparten un núcleo común de características altamente relevantes.

Arrhythmia: inestabilidad alta en todos los indicadores.
El CV de la accuracy alcanza el 57.4% y el del fitness el 34.5%, con valores individuales que van de 0.147 (semillas 44, 46, 48, 50, 51) a 0.566 (semilla 42). Esta bimodalidad es estadísticamente llamativa: 5 de 10

semillas producen accuracy por debajo del 0.20 en test, mientras las otras 5 superan el 0.43. No se trata de variación continua sino de convergencia a dos regímenes cualitativamente distintos: el GA encuentra un buen subconjunto o queda atrapado en un óptimo local de baja calidad. Kuncheva=0.143 confirma que los subconjuntos son casi tan diferentes como si fueran aleatorios (KI=0 implica solapamiento esperado por azar). Sin embargo, $|S^*|$ muestra CV del 9.7% (de 82 a 121), lo que indica que el GA selecciona tamaños similares independientemente de la calidad del subconjunto: el problema está en cuáles características, no en cuántas.

El núcleo de 15 características con frecuencia ≥ 0.8 identifica las variables con mayor consistencia multi-semilla. Con Kuncheva tan bajo, la existencia de un núcleo de 15 entre 279 (5.4%) sugiere que hay un pequeño conjunto de variables altamente informativo que el GA detecta en la mayoría de las ejecuciones, pero que por sí solo no es suficiente para lograr alta accuracy con GNB.

Gisette: paradoja de estabilidad fenotípica sin estabilidad genotípica.

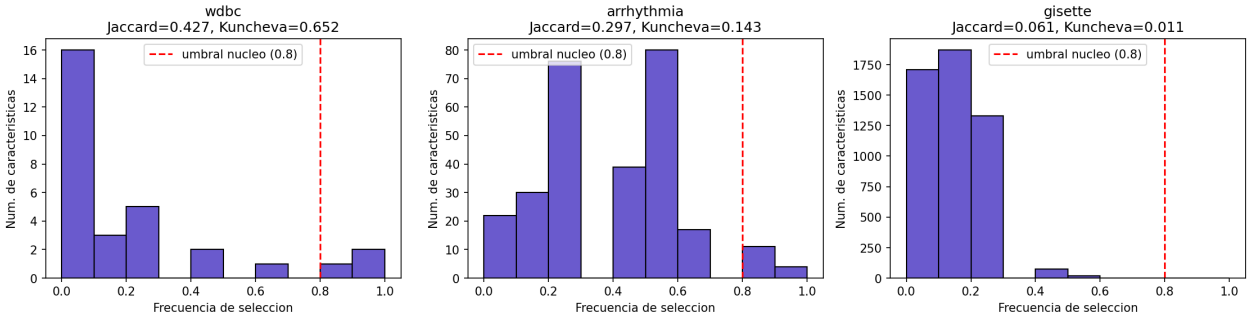


Figure 8: Histogramas de frecuencia de selección por característica en $R = 10$ ejecuciones, para cada dataset. La línea discontinua roja marca el umbral del núcleo estable ($f \geq 0.8$). *WDBC* (izquierda): distribución bimodal —la mayoría de características tiene frecuencia baja (< 0.2) y 3 superan el umbral, formando el núcleo estable. *Arrhythmia* (centro): distribución ancha y uniforme entre 0 y 0.8, con solo 15 características superando el umbral; el spread refleja la alta variabilidad entre semillas. *Gisette* (derecha): la gran mayoría de las 5000 características tiene frecuencia muy baja (< 0.2) y ninguna supera el umbral de 0.8, consistente con el Kuncheva≈0 y la redundancia masiva del dataset.

Gisette presenta el perfil más inusual (Figura~8, panel derecho): CV(acc)=0.7% y CV(fitness)=0.6% indican una estabilidad fenotípica casi perfecta —el GA converge siempre a una solución de calidad prácticamente idéntica— pero Jaccard=0.061 y Kuncheva=0.011 (próximo a 0, solapamiento aleatorio) muestran que los subconjuntos S^* son casi completamente distintos entre semillas, y el núcleo estable es vacío (ninguna característica aparece en ≥ 8 de 10 ejecuciones). La razón es la **redundancia masiva** del dataset: Gisette contiene 5000 características de las cuales la mayoría son ruido, pero entre las relevantes existe alta colinealidad (son transformaciones de los mismos dígitos). El GA puede seleccionar 483–571 características muy distintas que codifican la misma información discriminativa. Esta propiedad —alta estabilidad en calidad, baja estabilidad en identidad— es característica de problemas con muchas soluciones equivalentes en el espacio de búsqueda.

---

5. Hito 11: Validación con LinearSVC

5.1 Objetivo y metodología

La validación del subconjunto S^* con un clasificador distinto al usado durante el entrenamiento del GA (GNB) responde a la pregunta: **¿son las características seleccionadas genuinamente informativas, o el GA ha encontrado un subconjunto que solo funciona bien con GNB?** (**[D1]** §5.2; Kohavi & John,

1997). Adicionalmente, evaluar S^* sobre un test *held-out* evita el sesgo de selección documentado por Ambroise & McLachlan (2002): reportar la accuracy sobre las instancias ya vistas por el GA durante la búsqueda produce estimaciones optimistamente sesgadas (**[D4]** §4.2).

El protocolo es el siguiente: 1. Se toma el subconjunto S^* encontrado por el GA con semilla 42 sobre el split holdout. 2. Se entrena un **LinearSVC** (con `StandardScaler` en pipeline, `dual='auto'`, `max_iter=5000`) sobre `X_train[:, S^*]`, `y_train`. 3. Se evalúa sobre `X_test[:, S^*]`, `y_test`. 4. Se compara la accuracy de LinearSVC con la de GNB entrenado sobre el mismo subconjunto y evaluado sobre el mismo test.

El LinearSVC es un clasificador de margen máximo (SVM lineal) que realiza suposiciones radicalmente distintas a GNB: no asume independencia de características ni distribución gaussiana, sino que busca el hiperplano separador óptimo. La mejora con LinearSVC sobre GNB sobre el mismo S^* evidencia que la información capturada no es específica del clasificador base.

5.2 Resultados

Dataset	$\lvert S^* \rvert$	Reducción	GNB test	LinearSVC test	Mejora SVC
WDBC	6/30	80.0%	0.9474	0.9649	+0.0175
Arrhythmia	110/279	60.6%	0.5662	0.6250	+0.0588
Gisette	483/5000	90.3%	0.9210	0.9390	+0.0180

(Fuente: `results/validacion_linearsvc.csv`, `results/escalabilidad_algoritmica.csv`)

5.3 Interpretación

Los tres datasets muestran mejora con LinearSVC, lo que confirma que los subconjuntos encontrados por el GA codifican información discriminativa independiente del clasificador base.

WDBC (+1.75 pp): ambos clasificadores operan cerca del techo de rendimiento del dataset (0.947–0.965). La mejora es estadísticamente marginal; WDBC es un dataset de baja dificultad para clasificadores lineales, y la selección de solo 6 variables concentra la información esencial.

Arrhythmia (+5.88 pp): es la mejora más pronunciada en términos absolutos. GNB asume independencia condicional entre características, lo que es especialmente problemático en ECG donde las variables de distintos canales están correlacionadas. LinearSVC no impone esta restricción y aprovecha mejor las correlaciones dentro del subconjunto de 110 variables seleccionadas. Aun así, la accuracy del 62.5% sobre 13 clases desequilibradas (siendo la distribución de clases muy sesgada hacia la clase 1 con 245/452 muestras) refleja la dificultad inherente del problema.

Gisette (+1.80 pp): la mejora es similar a WDBC en magnitud. Con 483 características seleccionadas (de 5000), LinearSVC encuentra un hiperplano separador de alta calidad. El valor de 0.939 es notable considerando que el GA utiliza GNB para guiar la búsqueda: demuestra que las características con mayor información en el sentido naive-bayes son también las que permiten el mejor margen de separación lineal, consistente con la naturaleza del dataset (dígitos 4 y 9 en alta dimensión con variables mayoritariamente irrelevantes).

---

6. Hito 12: Análisis de Escalabilidad Paralela

La escalabilidad de la evaluación paralela del fitness se estudia desde dos perspectivas complementarias de la teoría de la computación paralela: (i) **escalado fuerte** (*strong scaling*), donde el tamaño del problema permanece constante y se incrementa el número de unidades de cómputo —marco de la ley de Amdahl (Amdahl, 1967)—; y (ii) **escalado débil y amortización del overhead** (*weak scaling*), donde los recursos paralelos permanecen fijos y se incrementa el tamaño del problema —marco de la ley de Gustafson (Gustafson, 1988)—. La primera medida (§6.1–6.3) varía el número de particiones con población fija; la segunda (§6.4–6.5) varía el tamaño de población con particiones fijas. Ambas miden el tiempo de una evaluación completa

de la población como mediana de $N_{\text{rep}} = 3$ repeticiones, robusta ante los *outliers* de la recolección de basura (GC) de la JVM.

6.1 Protocolo: speed-up por número de particiones (escalado fuerte)

Se mide el speed-up de la evaluación paralela del fitness con Spark RDD frente a la evaluación en serie (Python puro), **sin ejecutar el ciclo evolutivo completo**. Se usa una población fija de P cromosomas generados con `np.random.seed(42)` —el P propio de cada dataset— y se varía el número de particiones de 1 a 8. Cada configuración se repite 3 veces y se reporta la mediana. Los datos se exportan en `*_speedup.csv` por dataset.

$$\text{Speed-up}(n) = \frac{t_{\text{serie}}}{t_{\text{Spark}}(n)} \qquad \text{Eficiencia}(n) = \frac{\text{Speed-up}(n)}{n}$$

6.2 Resultados: speed-up por número de particiones

Dataset	t_{serie} (s)	Spark 1	Spark 2	Spark 4	Spark 8
WDBC ($D = 30$)	0.111	0.95×	1.67×	2.22×	2.10×
Arrhythmia ($D = 279$)	0.272	0.84×	1.50×	2.21×	**2.32**×
Gisette ($D = 5000$)	4.049	0.84×	1.24×	**1.43**×	1.22×

Dataset	Efic. 2 part.	Efic. 4 part.	Efic. 8 part.	Óptimo
WDBC	83.7%	55.4%	26.3%	4 particiones
Arrhythmia	74.8%	55.3%	29.0%	8 particiones
Gisette	61.8%	35.9%	15.3%	4 particiones

(Fuente: `results/wdbc_speedup.csv`, `results/arrhythmia_speedup.csv`, `results/gisette_speedup.csv`)

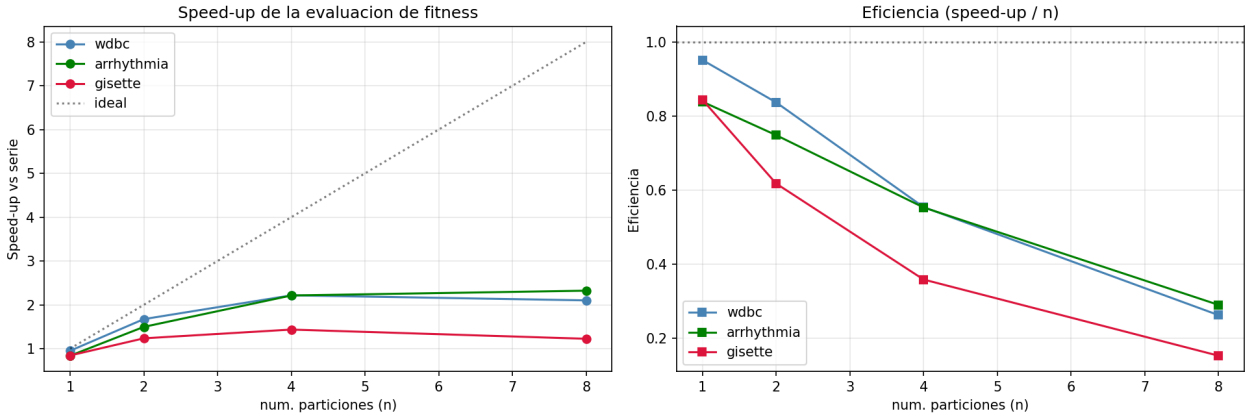


Figure 9: Speed-up (izquierda) y eficiencia (derecha) de la evaluación paralela del fitness con Spark RDD frente a ejecución en serie, para los tres datasets y 1, 2, 4 y 8 particiones. La línea discontinua en el panel izquierdo representa el speed-up ideal (lineal). *WDBC* (azul): speed-up máximo a 4 particiones ($2.22\times$) con descenso a 8 por granularidad excesiva. *Arrhythmia* (verde): speed-up creciente hasta 8 particiones ($2.32\times$, el mejor absoluto). *Gisette* (rojo): speed-up muy por debajo de los otros dos y con caída de 4 a 8 particiones ($1.43\times \rightarrow 1.22\times$) por el aviso de tarea de tamaño excesivo (1121 KiB). La eficiencia decrece en los tres datasets conforme aumentan las particiones, sin superar el 84% incluso en el punto óptimo.

6.3 Análisis por dataset

La Figura~9 muestra las curvas de speed-up y eficiencia. **WDBC:** con $t_{\text{serie}} = 0.111$ s para $P = 50$ individuos, el coste unitario de evaluación es de ~2.2 ms/individuo. El overhead fijo de Spark (serialización,

scheduling, recolección) es del mismo orden de magnitud, lo que explica que el speed-up con 1 partición sea ya inferior a 1 ($0.95\times$). El máximo se obtiene a 4 particiones ($2.22\times$); pasar a 8 reduce ligeramente el speed-up ($2.10\times$) por granularidad excesiva: con solo $P/8 \approx 6$ individuos por partición, el overhead por tarea supera el beneficio del paralelismo.

Arrhythmia: con cromosomas de $D = 279$ bits el coste por evaluación es mayor (~2.7 ms/individuo), permitiendo que el overhead de Spark sea más tolerable. El speed-up crece monotónicamente de 1 a 8 particiones, con el máximo en 8 ($2.32\times$). La eficiencia a 8 particiones (29.0%) es baja pero el speed-up es el mejor absoluto entre los tres datasets en escenario de 8 cores.

Gisette: anomalía en 8 particiones. El speed-up cae de $1.43\times$ (4 particiones) a $1.22\times$ (8 particiones), invirtiendo la tendencia esperada. La causa es el tamaño de los cromosomas: con $D = 5000$ y representación Python de enteros, cada cromosoma ocupa ~40 KB; una partición de $P/8 \approx 13$ cromosomas supera la recomendación de Spark de 1 MB por tarea (se registra aviso: *"Stage X contains a task of very large size (1121 KiB)"*). El overhead de serialización de las tareas a 8 particiones supera el beneficio del paralelismo adicional. La solución implementada en `run_ga` —compactar cromosomas a `int8` antes de paralelizar— reduce el tamaño a ~5 KB/cromosoma, pero el umbral de 1 MB/tarea se sigue superando con particiones pequeñas y cromosomas grandes.

6.4 Protocolo y resultados: speed-up por tamaño de población (escalado débil)

Complementando el análisis anterior, se fija el número de particiones al número de cores disponibles ($n = N_{\text{cores}} = 8$, el grano de paralelismo del entorno `local[*]`) y se varía el tamaño de población $P \in \{50, 100, 200, 500\}$. Para cada P se mide el tiempo de una evaluación completa de la población en serie (t_{serie}) y en paralelo (t_{Spark}), repitiendo $N_{\text{rep}} = 3$ veces y reportando la mediana. Mientras §6.1–6.3 hacen crecer los recursos sobre una carga constante, aquí la carga crece sobre recursos constantes, exponiendo cómo se amortiza el coste fijo de Spark. Los datos se exportan en `*_speedup_poblacion.csv`.

Dataset	$P = 50$	$P = 100$	$P = 200$	$P = 500$
WDBC ($D = 30$)	1.71×	2.53×	3.30×	**4.10**×
Arrhythmia ($D = 279$)	2.11×	2.79×	3.58×	**4.03**×
Gisette ($D = 5000$)	0.92×	**1.83**×	1.56×	1.61×

Speed-up vs. serie con $n = 8$ particiones fijas. Eficiencia (= speed-up$/8$): WDBC $0.21 \to 0.51$, Arrhythmia $0.26 \to 0.50$, Gisette 0.11–0.23 a lo largo del rango de P.

(Fuente: `results/wdbc_speedup_poblacion.csv`, `results/arrhythmia_speedup_poblacion.csv`, `results/gisette_speedup_poblacion.csv`)

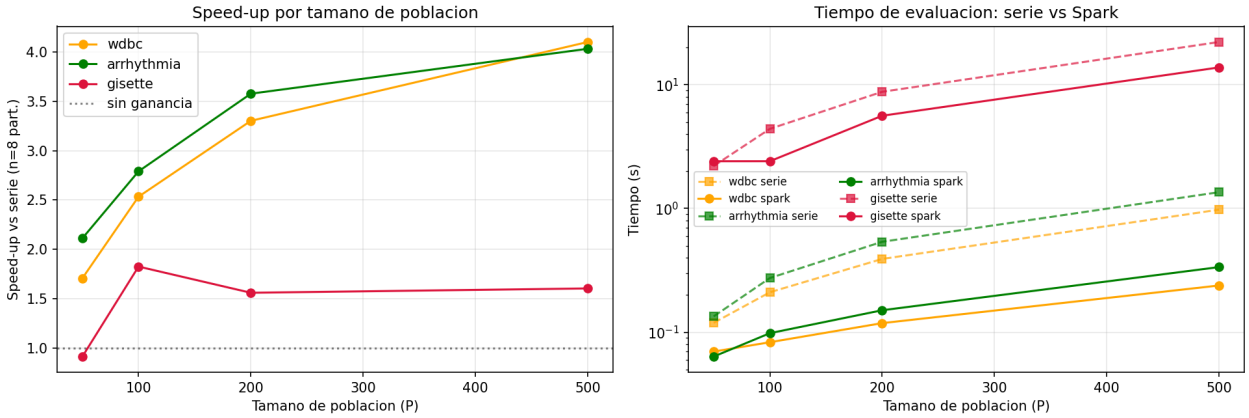


Figure 10: Escalado por tamaño de población con $n = 8$ particiones fijas. *Izquierda:* speed-up frente a la serie en función de P; la línea discontinua marca el umbral de ganancia nula ($1\times$). WDBC (azul) y Arrhythmia (verde) crecen monótonamente hacia el límite $n = 8$ a medida que el overhead fijo se amortiza; Gisette (rojo) parte por debajo de $1\times$ ($P = 50$, la versión paralela es más lenta que la serie) y se estanca en torno a $1.6\times$. *Derecha:* tiempos de evaluación serie (línea discontinua) y Spark (línea continua) en escala logarítmica; los tiempos serie crecen linealmente con P mientras que los de Spark crecen más despacio en WDBC/Arrhythmia (de ahí el speed-up creciente) pero casi en paralelo en Gisette (sin amortización).

6.5 Análisis: amortización del overhead

El tiempo de la evaluación paralela admite un modelo aditivo sencillo:

$$t_{\text{Spark}}(P) \approx T_o + \frac{P\,\tau}{n}, \qquad t_{\text{serie}}(P) = P\,\tau,$$

donde τ es el coste medio por evaluación individual y T_o el overhead fijo de Spark por ronda (planificación de tareas, serialización, *broadcast* de los datos y recolección de resultados). El speed-up resultante es

$$S(P) = \frac{P\tau}{T_o + P\tau/n} = \frac{n}{1 + \dfrac{n\,T_o}{P\,\tau}} \xrightarrow[P\to\infty]{} n.$$

El término $nT_o/(P\tau)$ es el overhead relativo, que decrece al aumentar P: a mayor carga útil por ronda, menor peso proporcional del coste fijo. Esta es la lógica del escalado débil de Gustafson (1988), dual de la ley de Amdahl: mientras Amdahl acota el speed-up con problema fijo, Gustafson muestra que el speed-up útil crece con el tamaño del problema cuando este satura los recursos disponibles.

WDBC y Arrhythmia confirman el modelo de amortización (Figura~10). El speed-up crece monótonamente con P: de $1.71\times$ ($P = 50$) a $4.10\times$ ($P = 500$) en WDBC, y de $2.11\times$ a $4.03\times$ en Arrhythmia. El coste por individuo es estable —verificable como $t_{\text{serie}}/P \approx 2.0$ ms en WDBC y 2.7 ms en Arrhythmia, constante en todo el rango—, de modo que el crecimiento del speed-up se explica íntegramente por la dilución del overhead fijo T_o sobre una carga creciente. Ambos datasets se aproximan asintóticamente al límite $n = 8$ sin alcanzarlo en $P = 500$, coherente con un T_o no despreciable y con un coste de planificación que crece levemente con el número de tareas.

Gisette está limitado por comunicación, no por amortización. Su speed-up no crece monótonamente: parte de $0.92\times$ en $P = 50$ —la versión paralela es *más lenta* que la serie—, salta a $1.83\times$ en $P = 100$ y se estanca en torno a $1.6\times$ para $P \geq 200$, sin tendencia ascendente. Aunque el coste por individuo es elevado ($\tau \approx 44$ ms), el overhead no es constante: con $D = 5000$ cada cromosoma compactado a `int8` ocupa $ $5 KB, y el volumen de datos serializado por ronda crece con P, por lo que T_o incorpora un término proporcional a P que cancela la amortización. Es el mismo fenómeno de tarea de tamaño

excesivo descrito en §6.3: Gisette opera en un régimen *communication-bound*, donde el cuello de botella es el transporte de datos y no el cómputo, y su eficiencia (≤ 0.23) es la más baja de los tres datasets en todo el rango de P.

6.6 Interpretación global

Ningún dataset alcanza la eficiencia ideal de 100% (speed-up lineal). Esto es esperado en el entorno `local[*]` de una sola máquina por tres razones: (1) ausencia de overhead real de red, que favorece la ejecución en serie en entorno local; (2) overhead de la JVM de Spark (scheduling, serialización, colección); (3) evaluaciones de fitness rápidas (ms–s) en comparación con clasificadores más costosos que justificarían mayor paralelismo.

La ley de Amdahl proporciona el marco teórico: si la fracción paralela es f (evaluación de fitness) y la secuencial es $1 - f$ (operadores genéticos en el Driver), el speed-up máximo teórico es $1/(1 - f + f/n)$. En los notebooks, el tiempo de los operadores es $t_{\text{ops}} \approx 0.001$ s/generación frente a $t_{\text{eval}} \approx 0.08$–$2.1$ s/generación, por lo que $f > 0.98$ y el límite teórico a 8 cores sería ~$7.7\times$. La brecha entre el límite teórico y el observado (~$2\times$) se atribuye íntegramente al overhead de Spark, y se reduciría en un cluster real con evaluaciones de mayor coste computacional (datasets más grandes, clasificadores más costosos).

Las dos perspectivas convergen en una misma conclusión y se refuerzan mutuamente. El escalado fuerte (§6.3) revela que el speed-up satura pronto —entre 2 y 4 particiones— porque, con carga fija, añadir particiones más allá del punto en que cada tarea aporta trabajo suficiente solo añade overhead (granularidad excesiva). El escalado débil (§6.5) muestra la cara complementaria: con particiones fijas, **aumentar la carga útil sí incrementa el speed-up** —de $\$\sim 1.7$–$2.1\times\$$ a $\$\sim 4\times\$$ en WDBC y Arrhythmia—, porque diluye el coste fijo T_o. Ambos resultados indican que el factor limitante en `local[*]` no es el grado de paralelismo sino la **relación cómputo/comunicación por tarea**: la paralelización con Spark RDD rinde cuando el trabajo por partición domina sobre su overhead de transporte. Esto explica por qué Gisette, pese a ser el dataset más costoso, obtiene el peor speed-up (su gran dimensionalidad lo desplaza al régimen *communication-bound*) y predice que el rendimiento mejoraría sustancialmente en un cluster con evaluaciones más caras por individuo —p.,ej., validación cruzada con más pliegues o clasificadores más costosos— donde τ crece sin que lo haga el volumen de datos transferido.

---

7. Tabla Resumen de Resultados del Mes 4

Dataset	D	Gens.	$\lvert S^* \rvert$	Reduc.	GNB	SVC	Jaccard	Kuncheva
WDBC	30	24	6	80.0%	0.9474	0.9649	0.4265	0.6524
Arrhythmia	279	100	110	60.6%	0.5662	0.6250	0.2975	0.1434
Gisette	5000	58	483	90.3%	0.9210	0.9390	0.0610	0.0108

(Fuente: `results/escalabilidad_algoritmica.csv`)

Las columnas Jaccard y Kuncheva corresponden al estudio de estabilidad con $R = 10$ semillas. Los valores de accuracy son sobre test held-out (30% del dataset, no visto durante el GA).

---

8. Artefactos Generados

Notebooks

Notebook	Descripción
`05_ga_full_arrhythmia.ipynb`	GA completo sobre Arrhythmia (dataset completo, seed=42, 100 gens)
`06_ga_full_gisette.ipynb`	GA completo sobre Gisette (dataset completo, seed=42, 44 gens)
`07_diversidad_escalabilidad.ipynb`	Holdout en 3 datasets; diversidad, estabilidad ($R = 10$), validación LinearSVC, speed-up

Módulos de código

Módulo	Descripción
src/ga_core.py	GA-wrapper reutilizable: run_ga(), operadores genéticos, criterio de parada dual
src/ga_metrics.py	Métricas de diversidad y estabilidad: Hamming, entropía, Jaccard, Kuncheva, frecuencia

Datos y gráficos (en results/)

Archivo	Contenido
ga_full_arrhythmia_experimento.json	Resumen completo: hardware, Spark, parámetros, resultados (Hito 7)
ga_full_gisette_experimento.json	Ídem para Gisette
ga_full_arrhythmia_series.csv	Series temporales por generación (dataset completo)
ga_full_gisette_series.csv	Ídem para Gisette
wdbc/arrhythmia/gisette_holdout_series.csv	Series por generación en holdout con métricas de diversidad
estabilidad_por_semilla.csv	Accuracy, fitness y $\|S^*\|$ por (dataset, semilla) $-$ $3 \times 10 = 30$ filas
estabilidad_resumen.csv	Media, std, Jaccard, Kuncheva, núcleo por dataset
validacion_linearsvc.csv	GNB vs. LinearSVC en test held-out
wdbc/arrhythmia/gisette_speedup.csv	Speed-up y eficiencia por número de particiones (población fija)
wdbc/arrhythmia/gisette_speedup_poblacion.csv	Speed-up y eficiencia por tamaño de población $P \in \{50, 100, 200, 500\}$ (particiones fijas)
escalabilidad_algoritmica.csv	Tabla consolidada: todas las métricas por dataset
*_diversidad.png	Curvas de diversidad y dispersión fenotípica por dataset
diversidad_comparativa.png	Comparativa del decaimiento de Hamming normalizado
estabilidad_frecuencias.png	Histogramas de frecuencia de selección por dataset
estabilidad_resumen.png	Barras con media ± std de accuracy, fitness y $\|S^*\|$
speedup_comparativo.png	Speed-up y eficiencia vs. número de particiones
speedup_poblacion_comparativo.png	Speed-up y tiempos serie/Spark vs. tamaño de población

---

9. Limitaciones y Trabajo Futuro

Limitaciones identificadas

1. **Inestabilidad de Arrhythmia:** la alta variabilidad entre semillas (CV acc = 57.4%) indica que el GA no es robusto en este dataset con los parámetros actuales. Aumentar P, G_{max} o ajustar la presión selectiva podría reducir la variabilidad, a costa de mayor tiempo de cómputo.
2. **Holdout no estratificado en Arrhythmia:** la función split_holdout() cae al split no estratificado cuando las clases más raras (2–5 muestras) impiden la estratificación, lo que puede producir splits donde algunas clases quedan fuera del conjunto de entrenamiento. Este es el factor principal que explica la disparidad entre el fitness de CV en entrenamiento y la accuracy en test para ciertas semillas.
3. **Speed-up limitado en entorno local:** el speed-up por número de particiones (máx. $2.32\times$ en Arrhythmia, 8 cores) refleja el overhead de Spark en local[*]. No obstante, el estudio por tamaño de población (§6.4–6.5) demuestra que este límite no es intrínseco al grado de paralelismo sino a la carga por tarea: con $P = 500$ el speed-up sube a $\approx 4\times$ en WDBC y Arrhythmia por amortización del overhead fijo. En un cluster real con evaluaciones de mayor coste por individuo el speed-up sería sustancialmente mayor; Gisette, por su parte, requeriría reducir el coste de comunicación (formato columnar, particionado de datos) para salir de su régimen *communication-bound*.
4. **Núcleo de Gisette vacío:** la ausencia de características con frecuencia ≥ 0.8 en Gisette impide identificar un conjunto de variables interpretable como “relevantes”. Esto es una propiedad del dataset (alta redundancia), no del algoritmo.

Siguiente paso

Con el estudio experimental completo, el mes 5 se dedicará a la redacción de la memoria del TFM, integrando los resultados de los cuatro meses de trabajo.

Referencias

Documentos internos del proyecto

Ref.	Fichero	Secciones relevantes
[D1]	`diseño_GA.md`	Fitness §2, operadores §3, GNB §5.1, LinearSVC §5.2, parámetros §6
[D2]	`arquitectura_spark_rdd_ga.md`	Driver/Workers §2.1, broadcast §2.5
[D4]	`plan_diversidad_escalabilidad.md`	Marco conceptual y plan experimental: métricas de diversidad §1.2, índice de Kuncheva §3.2, holdout y sesgo de selección §4

Notebooks

Ref.	Fichero	Contenido
[N5]	`05_ga_full_arrhythmia.ipynb`	GA completo Arrhythmia, dataset completo, seed=42
[N6]	`06_ga_full_gisette.ipynb`	GA completo Gisette, dataset completo, seed=42
[N7]	`07_diversidad_escalabilidad.ipynb`	Diversidad, estabilidad, validación, speed-up — 3 datasets

Archivos de resultados

Ref.	Fichero	Métricas
[R1]	`escalabilidad_algoritmica.csv`	Tabla consolidada: D, gens, $\lvert S^* \rvert$, reducción, accuracy GNB/SVC, Jaccard, Kuncheva, tiempos
[R2]	`estabilidad_por_semilla.csv`	30 filas: (dataset, semilla, $\lvert S^* \rvert$, fitness, acc_gnb_test)
[R3]	`estabilidad_resumen.csv`	Media ± std + índices de estabilidad por dataset
[R4]	`validacion_linearsvc.csv`	GNB vs. LinearSVC en test held-out
[R5]	`*_speedup.csv`	Speed-up y eficiencia por número de particiones (3 datasets)
[R6]	`*_speedup_poblacion.csv`	Speed-up y eficiencia por tamaño de población (3 datasets)

Bibliografía

Aalaei, S., Shahraki, H., Rowhanimanesh, A., & Eslami, S. (2016). Feature selection using genetic algorithm for breast cancer diagnosis. *Journal of AI and Data Mining*, 4(2), 185–194.

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Conference Proceedings*, 30, 483–485.

Ambroise, C., & McLachlan, G. J. (2002). Selection bias in gene extraction on the basis of microarray gene-expression data. *PNAS*, 99(10), 6562–6566.

Guyon, I., & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.

Guyon, I., Gunn, S., Ben-Hur, A., & Dror, G. (2004). Result analysis of the NIPS 2003 feature selection challenge. *Advances in Neural Information Processing Systems*, 17.

Guvenir, H. A., Acar, B., Demiroz, G., & Cekin, A. (1997). A supervised machine learning algorithm for arrhythmia analysis. *Computers in Cardiology*, 24, 433–436.

Gustafson, J. L. (1988). Reevaluating Amdahl's law. *Communications of the ACM*, 31(5), 532–533.

Huang, C., Davis, L. S., & Townshend, J. R. G. (2020). An assessment of support vector machines for land cover classification. *International Journal of Remote Sensing* [citado para el contexto de GNB en GA-wrapper].

Kohavi, R., & John, G. H. (1997). Wrappers for feature subset selection. *Artificial Intelligence*, 97(1–2), 273–324.

Kuncheva, L. I. (2007). A stability index for feature selection. *Proc. 25th IASTED Int. Conf. on Artificial Intelligence and Applications*, 421–427.

Mustaqeem, A., Anwar, S. M., & Majid, M. (2018). Multiclass classification of cardiac arrhythmia using improved feature selection and neural network. *Scientific Programming*, 1–10.

Rashid, A. N. M. B., & Abdullah, S. (2014). Feature selection using GA with NB: A review. *IOSR Journal of Computer Engineering*, 16(3), 67–72.

Siedlecki, W., & Sklansky, J. (1989). A note on genetic algorithms for large-scale feature selection. *Pattern Recognition Letters*, 10(5), 335–347.